Task Board Application - Hiring Assignment

Objective:
Build a small production-ready task board application demonstrating secure authentication,
role-based authorization, clean architecture, database design, and modern frontend practices.

Submission deadline:
The candidate has a maximum of 60 hours from receiving the assignment email to submit the repository.

Required Technology Stack

Area	Technology
Frontend	Next.js App Router, TypeScript, Tailwind CSS, Headless UI, TipTap
Backend	NestJS, TypeORM, PostgreSQL
Runtime	Node.js 20+
Database	PostgreSQL using Docker Compose

Repository Architecture

The solution must be implemented as a Monorepo.

Recommended structure:

task-board/
- frontend/ Next.js application
- backend/ NestJS application
- docs/ architecture and development documentation
- docker-compose.yml
- README.md
- AI development instructions

Frontend and backend responsibilities must remain clearly separated.

Application Architecture

The browser must communicate only with Next.js.

Requirements:
- Implement a proxy layer such as /api/v1/* from Next.js to NestJS.
- Access tokens must be stored in HttpOnly cookies.
- Database schema must be created only through migrations.
- TypeORM synchronize must remain disabled.

Authentication and Authorization

The application must support three roles:

Owner:
- Full access.
- Can manage users.
- Can disable users.
- Can change user roles.

Manager:
- Can create, edit, delete, and assign any task.
- Cannot manage users.

Member:
- Can only access tasks created by them or assigned to them.
- Cannot assign tasks to others.
- Cannot delete other users' tasks.

Security rules:
- Authorization must be enforced inside NestJS APIs.
- UI hiding is not considered security.
- Disabled users cannot login.
- Existing sessions/tokens of disabled users must stop working.
- The last active owner cannot be disabled or downgraded.

Seed Requirements

Initial users must be created through database seed scripts only.

The seed process must create:
- One active owner user.
- One manager user.
- One member user.

Additional requirements:
- Public registration is not required.
- Owner creation through UI is not required.
- Seed commands and credentials must be documented in README.
- Passwords must use argon2 or bcrypt.
- Seed execution must safely handle existing records.

Task Management

Task fields:
- Title
- Status (todo, in_progress, done)
- Priority

- Assignee
- Due date
- Description

Description requirements:
- Stored as TipTap JSON in PostgreSQL jsonb.
- HTML must not be the source of truth.
- Empty documents are valid.
- API validates allowed node types.
- Supported formatting: bold, italic, headings, lists, blockquote, links.
- Links must allow only http and https.
- Read-only rendering must use the same JSON document structure.

Frontend Requirements

The UI must be Persian and RTL.

Required:
- Headless UI based Dialog for create/edit forms.
- Headless UI Listbox or Menu for selections and actions.
- Accessible keyboard navigation and focus handling.
- Task filtering by status and assignee.
- No admin UI frameworks such as MUI or Ant Design.

Backend API Requirements

Required endpoints:

Authentication:
POST /auth/login
POST /auth/logout
POST /auth/refresh
GET /auth/me

Users:
GET /users
POST /users
PATCH /users/:id
(owner only)

Tasks:
GET /todos
POST /todos
GET /todos/:id
PATCH /todos/:id
DELETE /todos/:id

Backend requirements:
- DTO validation using class-validator.
- Reject unknown fields.
- Consistent error responses.
- Refresh tokens stored only as hashes.

Testing Requirements

Provide automated tests covering:

1. Member cannot view another user's tasks.
2. Member cannot modify users.
3. Disabled user cannot access APIs using an old token.

AI-Assisted Development Policy

Using AI coding assistants is allowed and encouraged.

Examples:
- Claude Code
- Cursor Agent
- OpenAI Codex
- GitHub Copilot

Positive evaluation points:
- Documented AI workflow.
- Project-specific AI instructions.
- Agent rules, skills, hooks, prompts, or similar documentation.
- Explanation of where AI was used and how generated code was reviewed.

The candidate remains responsible for architecture, security, quality, and final decisions.

Project Demonstration Video

Provide a Persian-language screen recording with a maximum duration of 5 minutes.

The video should demonstrate:
- Project overview.
- Architecture.
- Frontend/backend structure.
- Authentication and authorization flow.
- Task workflow.
- TipTap JSON implementation.
- Important technical decisions.

The goal is to evaluate understanding of the implementation.

README Delivery Requirements

README must include:

- Docker Compose setup.
- Database migration instructions.
- Seed instructions.
- Sample credentials.
- Architecture decisions.
- Environment setup.
- Known limitations or unfinished items.

The README must honestly describe remaining work if any.

Evaluation Criteria

1. Real authorization enforcement in NestJS.
2. Correct authentication, cookies, refresh tokens, and disabled-user handling.
3. Clean migrations and TypeORM relationships.
4. Correct TipTap JSON storage and restoration.
5. Accessible Headless UI implementation.
6. Clean monorepo architecture.
7. Quality of documentation and responsible AI-assisted development process.